

O tempo de reação do autoescalamamento é uma propriedade da plataforma? Um estudo de medição replicado do HPA do Kubernetes em Minikube e Amazon EKS

José Vitor Feitosa de Andrade

Escola Politécnica

Universidade de São Paulo

São Paulo, Brasil

NUSP 13682041

jose.vitor.0905@gmail.com

Fernando Frederice Miqueletti

Escola Politécnica

Universidade de São Paulo

São Paulo, Brasil

NUSP 12544340

fernando.miqueletti@usp.br

Resumo—Comparações de infraestrutura de nuvem feitas por profissionais costumam se apoiar em uma única execução por configuração. Este trabalho mostra, num cenário em que replicar é barato, o quanto esse desenho pode falhar — e mostra isso corrigindo uma conclusão nossa. Um estudo anterior implantou uma carga de trabalho Kubernetes idêntica, com autoescalamamento horizontal de Pods, em um *cluster* Minikube local e no serviço gerenciado Amazon EKS, executou cada implantação uma vez por integrante, observou que o ambiente mais rápido mudava entre as execuções e concluiu que o tempo de reação não era atribuível à plataforma. Este artigo mantém inalterados a implantação, a carga e o protocolo, e altera apenas o desenho experimental: vinte repetições não supervisionadas por ambiente, com a métrica primária derivada exclusivamente de instantes atribuídos pelo *cluster* — o `lastScaleTime` do próprio controlador e o `startedAt` do contêiner gerador de carga — de modo que nem o relógio do observador nem sua cadência de consulta entram na medição. Nas quarenta repetições, sem nenhuma perda, o tempo de reação foi de 40,8s no ambiente local contra 24,7s na nuvem (Mann-Whitney $U = 52,0$, $p = 1 \times 10^{-4}$; δ de Cliff = 0,74): existe um efeito de plataforma real, que o desenho anterior não tinha como resolver. E as mesmas distribuições explicam por que ele não tinha. Uma repetição sorteada de cada ambiente reproduz a ordenação correta em apenas 87 % dos casos, e as quatro observações relatadas anteriormente caem nos percentis 82, 10, 45 e 100 das distribuições aqui medidas — sorteios comuns que por acaso se inverteram. Registrar covariáveis junto de cada repetição permite testar explicações candidatas em vez de supô-las: nenhuma das cinco — latência de escrita no *control plane*, sua parcela de armazenamento, partida do gerador, utilização de pico ou deriva ao longo da campanha — prediz quais repetições locais foram lentas, apesar de a latência de escrita local variar de 114ms a 2534ms. Já tudo o que é fixado por declaração se reproduziu com variação de poucos por cento. Concluímos que o resultado lógico do autoescalamamento pode ser validado num *cluster* local, que sua latência não pode, e que a aritmética de uma comparação de execução única com $A_{12} = 0,87$ deveria desencorajar a prática muito além deste experimento.

Abstract—Practitioner comparisons of cloud infrastructure routinely rest on a single execution per configuration. We show, in a setting where replication is cheap, how badly that design can fail — and we show it by correcting a conclusion of our

own. An earlier study deployed an identical Kubernetes workload with the Horizontal Pod Autoscaler on a local Minikube cluster and on managed Amazon EKS, executed each deployment once per author, and observed that the faster environment changed between executions; it concluded that autoscaling reaction time was not attributable to the platform. This paper keeps the deployment, the workload and the load protocol unchanged, and changes only the experimental design: twenty unattended repetitions per environment, with the primary metric derived exclusively from timestamps assigned by the cluster. Across forty repetitions with no attrition, reaction time averaged 40.8s locally against 24.7s in the cloud (Mann-Whitney $U = 52.0$, $p = 1 \times 10^{-4}$; Cliff's $\delta = 0.74$): a real platform effect that the earlier design could not resolve. A single repetition drawn from each environment reproduces the correct ordering only 87 % of the time, and the four previously reported observations sit at the 82nd, 10th, 45th and 100th percentiles of the distributions measured here. None of five recorded covariates predicts which local repetitions were slow. Everything fixed by declaration replicated to within a few percent.

Index Terms—Computação em nuvem, Kubernetes, autoescalamamento, *Horizontal Pod Autoscaler*, elasticidade, reprodutibilidade, metodologia de medição, Amazon EKS.

I. INTRODUÇÃO

A elasticidade rápida é uma das cinco características essenciais da computação em nuvem na definição do NIST [1]: recursos devem ser provisionados e liberados automaticamente, de forma proporcional à demanda. No Kubernetes, o mecanismo que entrega essa propriedade na camada de aplicação é o *Horizontal Pod Autoscaler* (HPA) [2], uma malha de controle que periodicamente compara uma métrica observada com uma meta e ajusta o número de réplicas de uma carga de trabalho.

Quem escolhe entre um *cluster* local de desenvolvimento e um serviço gerenciado de nuvem pergunta, naturalmente, se o autoescalamamento *se comporta* igual nos dois. A pergunta tem duas metades distintas, e confundi-las é fonte de uma quantidade surpreendente de mal-entendidos. A primeira metade é o

resultado *lógico*: para quantas réplicas o controlador converge, e se a utilização observada se estabiliza na meta configurada. A segunda é o resultado *temporal*: quanto tempo o sistema leva para reagir a um pico de demanda.

Nosso próprio estudo anterior sobre essa questão ilustra a armadilha. No trabalho intermediário desta disciplina, implantamos uma carga de trabalho idêntica — a imagem oficial `hpa-example`, com manifestos, *requests*, *limits*, meta de utilização e política de escalonamento idênticos — em um *cluster* Minikube local e em um *cluster* gerenciado Amazon EKS, e cada integrante do grupo executou o experimento completo de forma independente. O resultado lógico replicou perfeitamente: as quatro implantações convergiram para seis ou sete réplicas, com a utilização média de CPU estabilizando na meta de 50%. O resultado temporal não replicou. Na primeira execução, o EKS reagiu cerca de quatro vezes mais rápido que o Minikube (15 s contra 62 s); na segunda, realizada em outra máquina e em outra conta AWS com o mesmo protocolo, a ordenação *se inverteu*, com o Minikube reagindo cerca de duas vezes mais rápido que o EKS.

Com uma execução por ambiente, a análise tinha de parar ali. Duas observações bastam para mostrar que um resultado não é estável, mas não bastam para dizer qualquer coisa sobre a distribuição subjacente: não há média, não há dispersão, não há intervalo de confiança e não há teste. A conclusão natural — “a diferença não é atribuível à plataforma” — era uma afirmação sobre a *ausência* de consistência, não uma caracterização do fenômeno.

Este artigo fecha essa lacuna. Mantemos deliberadamente inalterados a implantação, a carga e o protocolo do estudo anterior, e mudamos apenas o desenho experimental: em vez de uma execução por ambiente, conduzimos uma campanha automatizada e não supervisionada de execuções repetidas, e tratamos o tempo de reação como uma variável aleatória em vez de um número. Revisamos também a *procedência* de cada instante medido, derivando a métrica primária exclusivamente de carimbos de tempo atribuídos pelo próprio *cluster*, de modo que nem o relógio local nem a cadência de consulta do observador possam contaminá-la.

As questões de pesquisa são:

- QP1** Qual é a *distribuição* do tempo de reação do HPA em cada ambiente, e ela difere entre um *cluster* local e um gerenciado por mais do que ruído de amostragem?
- QP2** Os dois ambientes diferem em *dispersão* — isto é, o ambiente local não é apenas mais lento ou mais rápido, mas menos *previsível*?
- QP3** Dadas as distribuições medidas, qual é a probabilidade de um desenho de execução única — o desenho do estudo anterior, e da maior parte dos relatos de profissionais — conduzir a cada conclusão possível?
- QP4** Quais partes do comportamento de autoescalamento *são* reprodutíveis entre ambientes e, portanto, podem legitimamente ser validadas num *cluster* local?

A contribuição é tripla: um método de instrumentação que torna a latência de autoescalamento mensurável sem confiar no relógio do observador (Seção IV-D); uma caracterização

empírica da distribuição do tempo de reação nos dois ambientes, com tratamento não paramétrico completo (Seção VI); e uma resposta quantificada a quão enganoso é um experimento de execução única neste cenário (Seção VI-F) — uma questão que generaliza bem além do HPA.

II. FUNDAMENTAÇÃO

A. A malha de controle do HPA

O HPA é uma malha de controle fechada executada pelo `kube-controller-manager` em um período fixo (`--horizontal-pod-autoscaler-sync-period`, 15 s por padrão). A cada iteração ele lê o valor corrente da métrica configurada na *Metrics API* e calcula o número desejado de réplicas como [3]

$$r_{\text{desejado}} = \left\lceil r_{\text{atual}} \times \frac{m_{\text{atual}}}{m_{\text{meta}}} \right\rceil \quad (1)$$

onde m é, em nossa configuração, a utilização média de CPU dos Pods expressa como fração de seu *request* de CPU. O resultado é limitado ao intervalo `[minReplicas,maxReplicas]` e ainda restringido pela cláusula *behavior*, que define janelas de estabilização e políticas de taxa de forma independente para subida e descida.

A Equação (1) é puramente declarativa: não contém nenhum termo que dependa da infraestrutura. É precisamente por isso que se espera que o resultado *lógico* do autoescalamento replique entre ambientes, e isso delimita a questão que este artigo aborda — tudo o que pode diferir entre um *cluster* local e um gerenciado está na *temporização* da malha, não em sua aritmética.

B. Onde a latência pode entrar

Entre o instante em que uma carga se torna limitada por CPU e o instante em que uma nova réplica existe, acumulam-se vários atrasos independentes:

- 1) **Aquisição da métrica.** O `cAdvisor` do kubelet amostra a CPU dos contêineres periodicamente; o *Metrics Server* [4] coleta dos kubelets em seu próprio período (15 s por padrão) e serve uma janela móvel curta. Um pico de CPU, portanto, só se torna visível ao HPA depois de até um período completo de coleta, mais a largura da janela sobre a qual a taxa é calculada.
- 2) **Sincronização do controlador.** O HPA avalia a Equação (1) uma vez por período de sincronização. No pior caso, a métrica se torna visível imediatamente após uma avaliação e espera mais um período inteiro.
- 3) **Estabilização e políticas de taxa.** Com `scaleUp.stabilizationWindowSeconds` igual a zero, como em nossa configuração, nenhum atraso adicional é imposto na subida; a política percentual ainda limita o quanto a contagem de réplicas pode crescer por intervalo.
- 4) **Atuação.** Uma vez escalado o Deployment, o escalonador precisa alocar os novos Pods e o kubelet precisa

Tabela I
RESULTADOS DO ESTUDO ANTERIOR, DE EXECUÇÃO ÚNICA. O
RESULTADO LÓGICO REPLICOU; O TEMPORAL NÃO.

Métrica	Execução E1		Execução E2	
	Minikube	EKS	Minikube	EKS
Réplicas no regime	6	6	6	7
CPU no regime	51 %	47 %	~50 %	~50 %
CPU de pico	153 %	237 %	166 %	n/d
Tempo de reação	62 s	15 s	~33 s	~60 s
Descida ao mínimo	7,6 min	6,8 min	13–14 min	9–10 min

iniciá-los — um atraso que depende do cache de imagens, da capacidade do nó e, em um *cluster* gerenciado, do escalonamento entre nós.

Apenas os itens 1 e 2 precedem a *decisão*. Essa distinção importa para a definição da métrica na Seção IV-D: medimos o tempo até a *decisão* do controlador, não até os novos Pods estarem prontos, e reportamos os atrasos de atuação separadamente. A quantização implicada pelos itens 1 e 2 também explica por que não se deve esperar que o tempo de reação seja normalmente distribuído — é uma quantidade limitada inferiormente e discretizada, o que motiva o tratamento não paramétrico da Seção V.

C. Reprodutibilidade em medição de sistemas

A preocupação metodológica deste artigo não é nova. É bem estabelecido que medições de desempenho em infraestrutura compartilhada estão sujeitas a variabilidade grande o bastante para inverter conclusões, e que experimentos de execução única são correspondentemente frágeis [7], [8]. A recomendação prática — relatar distribuições e tamanhos de efeito em vez de comparações pontuais, e usar testes não paramétricos quando a distribuição subjacente é desconhecida — é padrão na literatura de metodologia experimental [9]. O que este artigo acrescenta é uma instância concreta no domínio de orquestração em nuvem, onde o custo de replicar é quase nulo, a variabilidade é alta, e a literatura — inclusive nosso próprio trabalho anterior — rotineiramente relata execuções únicas.

III. O TRABALHO ANTERIOR E SUA LIMITAÇÃO

O trabalho intermediário implantou a carga descrita na Seção IV em Minikube e em Amazon EKS. Cada um dos dois autores executou o experimento completo de forma independente, em máquinas e contas AWS distintas, aplicando manifestos idênticos byte a byte e o mesmo protocolo de carga. Os resultados estão resumidos na Tabela I.

Dois aspectos da Tabela I motivam este artigo. Primeiro, as linhas que descrevem o resultado *lógico* são estáveis nas quatro implantações. Segundo, a linha do tempo de reação não é apenas ruidosa — a *ordenação* dos dois ambientes se inverte entre E1 e E2. Se apenas E1 tivesse sido realizada, o estudo teria concluído, com dados internamente consistentes e uma explicação mecanística plausível, que o *control plane* gerenciado reage quatro vezes mais rápido. Se apenas E2 tivesse sido realizada, teria concluído o oposto.

Tabela II
CONFIGURAÇÃO MANTIDA IDÊNTICA ENTRE AMBIENTES E CAMPANHAS.

Parâmetro	Valor
Imagem da aplicação	registry.k8s.io/hpa-example
<i>Request / limit</i> de CPU	200 m / 500 m
<i>Request / limit</i> de memória	64 Mi / 128 Mi
Versão da API do HPA	autoscaling/v2
Métrica	utilização de CPU, média
Meta	50 % do <i>request</i>
minReplicas / maxReplicas	1 / 10
scaleUp.stabilizationWindow	0 s
Política de scaleUp	100 % a cada 15 s
scaleDown.stabilizationWindow	300 s
Política de scaleDown	50 % a cada 60 s
Gerador de carga	um Pod busybox:1.28, laço sequencial
Duração da carga por repetição	240 s

O desenho do estudo anterior deixou também três fragilidades de instrumentação que este artigo remove. O tempo de reação era calculado a partir do relógio de parede no momento em que o operador lançava o gerador de carga, o que inclui o escalonamento e a partida do Pod — uma quantidade que difere sistematicamente entre um *cluster* local de nó único com cache de imagens quente e um *cluster* gerenciado de dois nós. O período de amostragem era de 5 s, grosseiro em relação ao período de 15 s do controlador. Por fim, as duas execuções usaram instrumentação heterogênea: E1 dispunha de um amostrador automatizado, enquanto E2 se baseou em observação direta, de modo que parte da diferença observada poderia ser de medição, não de fenômeno.

IV. DESENHO EXPERIMENTAL

A. O que é mantido constante

O sistema implantado é idêntico byte a byte ao do estudo anterior, e idêntico entre os dois ambientes. A Tabela II lista a configuração. Mantê-la fixa é deliberado: torna a campanha diretamente comparável tanto às execuções únicas anteriores quanto entre ambientes, de modo que qualquer diferença observada seja atribuível ao ambiente e não à especificação.

O gerador de carga merece comentário. É um único Pod executando `while sleep 0.01; do wget -q -O- http://php-apache; done` — um laço de requisições sequencial e de uma única linha de execução. É um gerador de carga fraco, e o estudo anterior o identificou como o fator que impediu qualquer dos *clusters* de alcançar o teto de dez réplicas. Nós o mantemos inalterado, porque o objeto de estudo aqui é a *reação* do controlador ao início da carga, não o limite de capacidade dos *clusters*; substituir o gerador tornaria a campanha incomparável a E1 e E2, que é justamente a comparação sobre a qual o artigo se constrói. A Seção VIII retoma as consequências dessa escolha.

B. Ambientes

O ambiente local é um *cluster* Minikube de nó único sobre o *driver* Docker, executando em um hospedeiro Windows 11

Tabela III
CARACTERÍSTICAS DOS AMBIENTES CONFORME REPORTADAS POR CADA *cluster* EM TEMPO DE EXECUÇÃO, E NÃO CONFORME SOLICITADAS NA CRIAÇÃO.

Característica	Minikube	Amazon EKS
Kubernetes (<i>API server</i>)	v1.35.1	v1.34.11-eks
Nós	1 (<i>control plane</i> e <i>trabalhador</i>)	2 trabalhadores
CPU alocável por nó	4000 m	1930 m
Memória alocável por nó	7,8 GiB	1,4 GiB
Tipo de instância	n/d (contêiner em Windows 11 + WSL2)	t3.small
SO do nó	Debian 12	Amazon Linux 2023
<i>Runtime</i> de contêiner	Docker 29.2.1	containerd 2.2.7
Zonas de disponibilidade	n/d	us-east-1b, us-east-1f
<i>Metrics Server</i>	<i>addon</i> do Minikube	<i>addon</i> gerenciado do EKS
Desvio de relógio cliente- <i>cluster</i>	0 s	-2 s

com WSL2. O ambiente de nuvem é um *cluster* gerenciado Amazon EKS criado com `eksctl` [6], com um *node group* gerenciado de duas instâncias `t3.small` em `us-east-1` — a mesma topologia do estudo anterior. As características efetivamente reportadas por cada *cluster* em tempo de execução, e não as solicitadas na criação, estão na Tabela III; a distinção importa, como a Seção VIII explica.

Duas entradas da Tabela III merecem comentário. O *cluster* local foi criado com `--cpus=2 --memory=4096`, mas reporta 4000 m de CPU alocável e 7,8 GiB de memória: o *driver* Docker Desktop não aplica esses limites, comportamento já observado no estudo anterior e aqui confirmado. O nó local é, portanto, *maior* que os dois nós de nuvem somados (4000 m contra 2×1930 m), o que importa na interpretação dos resultados ligados a capacidade. O desvio de relógio é reportado apenas por completude; a métrica primária é uma diferença entre dois carimbos de tempo atribuídos pelo *cluster* e não depende do relógio do cliente.

C. Protocolo de cada repetição

Uma campanha consiste em N repetições do protocolo abaixo, executadas sem supervisão por `scripts/10-run-campaign.sh`:

- 1) **Reinício.** Qualquer gerador de carga é removido e o Deployment é escalado para uma réplica.
- 2) **Verificação do estado de base.** A repetição não começa até que o HPA reporte uma réplica desejada, uma réplica pronta e utilização de CPU observada igual ou inferior a 10 % por três amostras consecutivas. Como `scaleUp.stabilizationWindowSeconds` é zero, o controlador desfaz uma redução manual enquanto o *Metrics Server* ainda reporta a CPU decaindo da repetição anterior; o laço, por isso, reaplica a escala até que a CPU medida tenha de fato decaído. Sem essa etapa, as repetições não seriam independentes.

- 3) **Amostragem.** Um amostrador registra o estado do HPA e do Deployment a cada 2 s, com compensação de deriva para que o intervalo efetivo não cresça quando chamadas individuais à API demoram mais.
- 4) **Carga.** O Pod gerador é submetido. O T_0 da repetição é o instante em que seu contêiner *efetivamente começou* a executar, não o instante da submissão (Seção IV-D).
- 5) **Deteção.** A primeira decisão de escala do controlador é detectada consultando `hpa.status.lastScaleTime` uma vez por segundo e tomando o primeiro valor que seja, ao mesmo tempo, diferente do valor anterior à carga e não anterior a T_0 .
- 6) **Manutenção e encerramento.** A carga é mantida por 240 s e o gerador é removido.
- 7) **Descida.** Nas três primeiras repetições de cada campanha a descida natural é observada até o fim; nas demais, o retorno a uma réplica é forçado, para manter a campanha em duração limitada. A janela de estabilização de 300 s faz uma descida natural completa custar cerca de dez minutos por repetição, o que mais que dobraria a campanha e, no ambiente de nuvem, seu custo.

D. Instrumentação: a procedência de cada instante

A escolha metodológica central deste artigo é que a métrica primária é calculada a partir de dois carimbos de tempo atribuídos pelo *cluster*, nunca pelo cliente que mede:

- T_0 é `pod.status.containerStatuses[0].state.running.startedAt` do gerador de carga — carimbado pelo kubelet quando o contêiner começou a executar.
- T_1 é `hpa.status.lastScaleTime` — carimbado pelo próprio controlador do HPA quando ele agiu.
- **Tempo de reação** := $T_1 - T_0$.

Como ambos os instantes se originam dentro do *cluster*, a medição é imune ao relógio local, ao tempo de ida e volta na rede e ao instante em que o amostrador por acaso perguntou. Isso não é um refinamento cosmético. No estudo anterior, T_0 era o relógio de parede do operador na submissão, o que embute silenciosamente o escalonamento e o *pull* da imagem do Pod no tempo de reação relatado — e esse atraso é exatamente o tipo de quantidade que difere entre um nó local quente e um *cluster* gerenciado de dois nós. Aqui ele é medido como variável separada (*partida do gerador*, o intervalo da submissão até o início do contêiner) e reportado de forma independente.

a) *Manter a amostragem simétrica:* Um obstáculo prático ameaçou reintroduzir justamente o defeito que nos propusemos a remover. O `kubeconfig` gerado por `aws eks update-kubeconfig` autentica através de um *exec credential plugin*: cada invocação do `kubectl` lança a AWS CLI para emitir um *token*. No hospedeiro de medição isso custou 2,8 s por chamada, de modo que um amostrador configurado para 2 s entregava, na prática, uma amostra a cada 5 s contra o EKS, enquanto entregava 2 s contra o Minikube. A métrica primária sobreviveria a isso — é derivada de carimbos atribuídos

pelo *cluster* e é indiferente a quando o observador pergunta — mas toda quantidade derivada da série temporal (utilização de pico, tempo até o regime permanente) teria sido medida em resoluções diferentes nos dois ambientes. Essa é exatamente a ameaça de instrumentação heterogênea que o estudo anterior identificou em si mesmo.

Substituímos então o *exec plugin* por um *kubeconfig* com um *token* previamente emitido, renovado em segundo plano a cada sete minutos porque os *tokens* do EKS têm vida curta. A latência por chamada caiu de 2,8 s para 0,51 s, e o período efetivo de amostragem passou a ser constante de 2 s nos dois ambientes. Os 0,51 s residuais são tempo de ida e volta e negociação TLS até *us-east-1*, que nenhuma mudança do lado do cliente remove, e são a razão pela qual os valores de latência do lado do cliente da Seção VI devem ser lidos como uma *diferença* entre escrita e leitura, e não em termos absolutos.

Duas verificações cruzadas acompanham a métrica primária. Primeiro, um tempo de reação derivado da consulta — a primeira amostra em que *desired* excede um — é registrado em paralelo; espera-se que seja ligeiramente maior e mais grosseiro, e qualquer divergência grande indicaria falha de instrumentação. Segundo, o desvio entre o relógio do cliente e o do *cluster* é medido no início de cada campanha criando um objeto descartável e comparando o *creationTimestamp* atribuído pelo servidor com o relógio local, e é reportado na Tabela III. Ainda que a métrica primária não dependa dele, publicar o desvio permite ao leitor limitar o erro de qualquer quantidade medida do lado do cliente.

V. MÉTODOS ESTATÍSTICOS

Toda a análise é feita por *scripts/11-analyze.py*, implementado usando apenas a biblioteca padrão do Python — sem *numpy*, sem *scipy*. A motivação é reprodutibilidade: o leitor consegue reexecutar a análise sem instalar nada, e cada estimador fica visível no código-fonte em vez de escondido atrás de uma chamada de biblioteca. A semente aleatória é fixa, de modo que os resultados de *bootstrap* e de permutação são reprodutíveis.

a) *Estatísticas descritivas*: Reportamos n , média, desvio-padrão amostral, coeficiente de variação, mínimo, quartis, máximo e desvio absoluto mediano (MAD) para cada métrica e ambiente.

b) *Estimação por intervalo*: Os intervalos de confiança para a média são calculados por *bootstrap* percentil não paramétrico com $B = 20\,000$ reamostragens, em vez da distribuição t . O tempo de reação é limitado inferiormente por zero e quantizado pelo período de sincronização do controlador (Seção II-B), de modo que normalidade não é uma suposição defensável com $n \approx 20$.

c) *Teste de hipótese*: Os dois ambientes são comparados com um teste de Mann–Whitney U bilateral, usando a aproximação normal com correções de continuidade e de empates. Empates são esperados e precisam ser corrigidos precisamente porque a medição é quantizada. Como segunda verificação, livre de suposições, executamos um teste de permutação sobre

a diferença de medianas, com 50 000 reetiquetagens sob a hipótese nula de permutabilidade.

d) *Dispersão*: A QP2 trata de variabilidade e não de posição, então a testamos diretamente: um teste de permutação sobre o logaritmo da razão entre os dois MADs. Essa é a formulação formal da afirmação de que um ambiente local é menos *previsível*, o que é distinto de — e não implicado por — ser mais lento em média.

e) *Tamanho de efeito*: Reportamos o δ de Cliff e a estatística A_{12} de Vargha–Delaney. São os tamanhos de efeito naturais aqui porque A_{12} tem leitura operacional direta neste experimento: é a probabilidade de uma repetição sorteada de um ambiente exceder uma repetição sorteada do outro — que é exatamente a pergunta que um estudo de execução única responde implicitamente, e portanto a base da Seção VI-F.

VI. RESULTADOS

As duas campanhas se completaram sem nenhuma perda: 20 repetições válidas por ambiente, 2849 e 2988 amostras de série temporal respectivamente, e nenhuma repetição perdida por *cluster* degradado ou por falha do gerador de carga. A Tabela IV traz as estatísticas descritivas de todas as métricas; o texto abaixo destaca o que importa para as questões de pesquisa.

A. A distribuição do tempo de reação (QP1)

O tempo de reação no ambiente local teve média de 40,8 s (DP = 18,5; mediana 33,5; IIQ 30,8–41,5; faixa 19–87; IC de 95 % da média [33,5; 49,2]). No ambiente gerenciado, média de 24,7 s (DP = 5,9; mediana 24,5; IIQ 21,8–27,2; faixa 14–36; IC [22,1; 27,3]). As distribuições empíricas aparecem na Fig. 2: a curva do EKS está deslocada à esquerda e é marcadamente mais íngreme.

A diferença é estatisticamente significativa e o efeito é grande. Um teste de Mann–Whitney bilateral dá $U = 52,0$, $z = -3,99$, $p = 1 \times 10^{-4}$; um teste de permutação sobre a diferença de medianas dá diferença de 9,0 s com $p = 4 \times 10^{-4}$. O δ de Cliff é 0,740 e $A_{12} = 0,870$: uma repetição sorteada do ambiente local é mais lenta que uma sorteada do ambiente gerenciado em 87 % de todos os pares possíveis.

Vale dizer isso sem rodeios, porque contradiz a conclusão do nosso próprio estudo anterior. *Existe* um efeito de plataforma sobre o tempo de reação, e o ambiente gerenciado é o mais rápido. O que faltava ao estudo anterior não era um argumento melhor, era amostra: com uma observação por ambiente, um efeito desse tamanho simplesmente não é resolvível contra tanto ruído.

B. Dispersão (QP2)

O ambiente local é mais variável por toda medida descritiva — DP de 18,5 s contra 5,9 s, coeficiente de variação de 0,45 contra 0,24, amplitude de 68 s contra 22 s. O teste formal, entretanto, não sustenta a afirmação: um teste de permutação sobre a razão dos desvios absolutos medianos resulta em razão de 1,33 com $p = 0,41$.

Tabela IV
ESTATÍSTICAS DESCRITIVAS DAS REPETIÇÕES (IC DE 95 % DA MÉDIA POR BOOTSTRAP, $B = 20000$).

Métrica	Ambiente	n	Média	DP	CV	Mediana	IIQ	IC 95%
Tempo de reação do HPA (s)	Minikube	20	40.8	18.5	0.45	33.5	30.8–41.5	[33.5, 49.2]
Tempo de reação do HPA (s)	EKS	20	24.7	5.9	0.24	24.5	21.8–27.2	[22.1, 27.3]
Tempo até o regime permanente (s)	Minikube	20	197.6	23.1	0.12	204.5	181.2–218.0	[187.6, 207.1]
Tempo até o regime permanente (s)	EKS	20	83.0	46.7	0.56	69.5	57.8–83.2	[65.8, 105.8]
Réplicas no regime permanente	Minikube	20	6.0	0.4	0.07	6.0	6.0–6.0	[5.9, 6.2]
Réplicas no regime permanente	EKS	20	6.7	0.6	0.09	7.0	6.0–7.0	[6.4, 6.9]
CPU de pico observada (%)	Minikube	20	176.5	37.5	0.21	167.0	159.8–179.2	[161.4, 193.6]
CPU de pico observada (%)	EKS	20	236.2	21.5	0.09	244.0	232.0–250.0	[226.1, 244.2]
Partida do gerador de carga (s)	Minikube	20	2.4	1.0	0.41	2.0	2.0–3.0	[2.0, 2.8]
Partida do gerador de carga (s)	EKS	20	1.8	0.4	0.25	2.0	1.8–2.0	[1.6, 1.9]
Retorno ao mínimo após a carga (s)	Minikube	3	453.0	6.9	0.02	449.0	449.0–455.0	[449.0, 461.0]
Retorno ao mínimo após a carga (s)	EKS	3	388.7	10.7	0.03	383.0	382.5–392.0	[382.0, 401.0]
Latência de leitura do <i>API server</i> (ms)	Minikube	20	108.8	3.4	0.03	108.0	106.8–111.2	[107.4, 110.3]
Latência de leitura do <i>API server</i> (ms)	EKS	20	547.4	53.6	0.10	519.5	516.2–550.5	[527.0, 571.7]
Latência de escrita no <i>control plane</i> (ms)	Minikube	20	464.8	575.5	1.24	197.5	147.5–617.8	[258.4, 743.6]
Latência de escrita no <i>control plane</i> (ms)	EKS	20	555.2	57.1	0.10	525.5	521.5–559.5	[532.5, 581.0]
Parcela de armazenamento em ms (escrita – leitura)	Minikube	20	355.9	576.1	1.62	92.0	33.2–511.5	[149.7, 630.2]
Parcela de armazenamento em ms (escrita – leitura)	EKS	20	7.8	14.2	1.81	6.0	2.8–8.0	[3.0, 14.8]

Tabela V
TESTES DE HIPÓTESE, MINIKUBE VS. EKS.

Teste	Estatística	p	Tamanho de efeito
Mann–Whitney U (tempo de reação)	$U = 52.0$, $z = -3.99$	0.0001	$\delta = 0.740$ (grande)
Permutação (diferença de medianas)	$\Delta = 9.0$ s	0.0004	–
Permutação (razão de dispersões)	razão de MAD = 1.33	0.4098	–

A discrepância é informativa, e não contraditória. O MAD é uma estatística robusta, e a distribuição local não é uniformemente espalhada: dezesseis de suas vinte observações caem entre 19 s e 43 s, faixa comparável à do ambiente gerenciado, enquanto quatro observações (62, 73, 76 e 87 s) formam uma longa cauda à direita. A dispersão robusta é, portanto, semelhante; o que difere é a presença da cauda. Concluimos que a QP2 é respondida negativamente como formulada — não conseguimos estabelecer diferença na dispersão típica — registrando que o ambiente local produziu valores extremos que o gerenciado nunca produziu.

C. O que as repetições não explicam

Como a campanha registrou covariáveis junto de cada repetição, as fontes dessa variabilidade podem ser testadas em vez de conjecturadas. Cinco explicações candidatas foram examinadas por correlação de Spearman contra o tempo de reação, com p -valores por permutação. No ambiente local, nenhuma delas é significativa: latência de escrita no *control plane* ($\rho = -0,33$, $p = 0,16$), sua parcela de armazenamento ($\rho = -0,33$, $p = 0,15$), partida do gerador de carga ($\rho = -0,33$, $p = 0,16$), CPU de pico ($\rho = 0,08$, $p = 0,72$) e índice da repetição, que revelaria deriva ao longo da campanha ($\rho = 0,28$, $p = 0,22$).

Esse resultado nulo é o achado mais útil da subseção. O *control plane* local estava demonstravelmente insalubre — sua latência de escrita teve coeficiente de variação de 1,24 e variou de 114 ms a 2534 ms, contra uma latência de leitura que se manteve entre 104 e 115 ms — e ainda assim essa instabilidade não prediz quais repetições reagiram devagar. A variabilidade do tempo de reação, portanto, não é sintoma de um ambiente degradado que uma máquina melhor removeria.

O mecanismo compatível com essas observações é a *fase*. Como descreve a Seção II-B, um pico de CPU precisa primeiro ser amostrado pelo kubelet, depois coletado pelo *Metrics Server* em seu próprio período, e depois lido pelo HPA em seu período de sincronização. O início da carga cai em um ponto arbitrário desses ciclos, e esse ponto sozinho pode responder por dezenas de segundos. A fase não é propriedade da plataforma, não é observável de fora e não é reproduzível — o que é precisamente a razão pela qual não se pode confiar a uma execução única a medição dessa grandeza.

No ambiente gerenciado, duas covariáveis alcançam significância, e ambas sobrevivem a uma correção de Bonferroni para os dez testes realizados. A CPU de pico correlaciona positivamente com o tempo de reação ($\rho = 0,687$, $p = 1,4 \times 10^{-3}$); interpretamos isso como causalidade invertida, e não como explicação, já que um controlador que decide mais tarde permite à utilização subir mais antes da primeira ampliação,

tornando um pico alto *consequência* de uma reação lenta. A latência de escrita correlaciona negativamente ($\rho = -0,767$, $p = 4 \times 10^{-4}$), o que não tem leitura causal que nos convença; o mecanismo plausível é que a sonda de latência roda imediatamente antes da carga e sua própria duração desloca a fase em que a carga começa. Que o instrumento perturbe a grandeza que mede é uma limitação que registramos na Seção VIII, não um achado.

D. Trajetória, convergência e o resultado lógico (QP4)

A Fig. 1 mostra as trajetórias de réplicas e a Fig. 4 a utilização sobre a qual o controlador realimentava. Ambos os ambientes convergem e se estabilizam na meta configurada, reproduzindo o resultado qualitativo do estudo anterior. Os detalhes quantitativos, porém, diferem mais do que aquele estudo poderia ter detectado.

O ambiente gerenciado assentou em mais réplicas: mediana 7 (média 6,7; faixa 6–8) contra mediana 6 (média 6,0; faixa 5–7) no local. Isso decorre da capacidade, e não do controlador: o nó local oferece 4000m de CPU alocável em um único nó, enquanto cada nó de nuvem oferece 1930m, e o mesmo gerador de carga sequencial produziu utilização de pico mais alta na nuvem (236 % contra 176 % do *request*). A convergência também foi bem mais rápida na nuvem — 83,0s até o regime permanente contra 197,6s no local — embora com espalhamento muito maior (CV de 0,56 contra 0,12).

A descida, medida em três repetições por ambiente, levou 453,0s (DP 6,9) no local e 388,7s (DP 10,7) na nuvem. Ambas são dominadas pela janela de estabilização de 300s configurada no manifesto, e ambas são notavelmente consistentes: a descida é governada por política declarada, e política se reproduz onde temporização não se reproduz.

E. Latência do lado do cliente e sua interpretação

Os dois valores de latência da Tabela IV precisam ser lidos em par. A latência de leitura do *API server*, que não toca o etcd, foi de 108,8ms no local e 547,4ms contra o EKS — esta última é tempo de ida e volta e negociação TLS até *us-east-1*, não lentidão do *cluster*. A parcela de armazenamento, obtida subtraindo a leitura da escrita, conta a história oposta: 355,9ms no local (mediana 92ms, máximo 2425ms) contra 7,8ms na nuvem (mediana 6ms). O caminho de armazenamento do *control plane* gerenciado é cerca de duas ordens de magnitude mais rápido e incomparavelmente mais estável. Medir apenas a latência de escrita levaria à conclusão de que os dois *control planes* são semelhantes (464,8ms contra 555,2ms), o que é exatamente o contrário.

F. Quão enganosa é uma execução única?

As distribuições medidas permitem avaliar diretamente o desenho do estudo anterior. Sorteando uma repetição de cada ambiente — que é exatamente o que um experimento de execução única faz — a probabilidade de cada conclusão possível é: **87,0 %** de o ambiente gerenciado parecer mais rápido, **12,2 %** de o local parecer mais rápido e 1,5 % de empate exato. Um

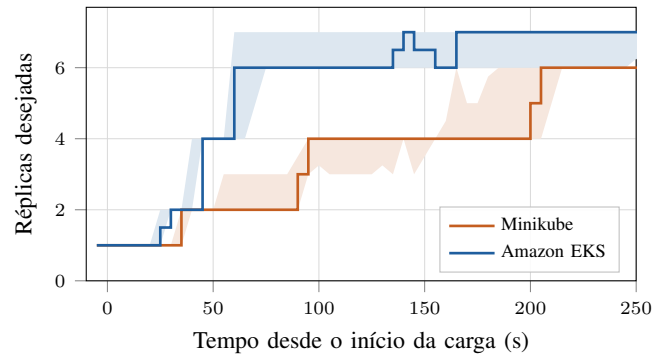


Figura 1. Trajetória de réplicas ao longo das repetições: mediana (linha) e amplitude interquartil (faixa) do número de réplicas desejadas, alinhadas pelo instante em que o contêiner do gerador de carga começou a executar.

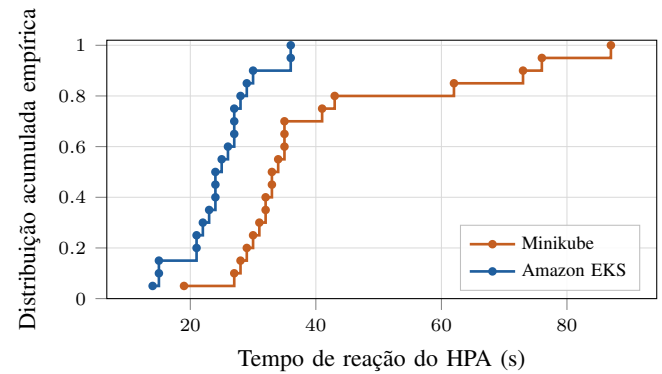


Figura 2. Distribuição acumulada empírica do tempo de reação do HPA. Todas as observações da campanha estão representadas; nenhum modelo distribucional é imposto.

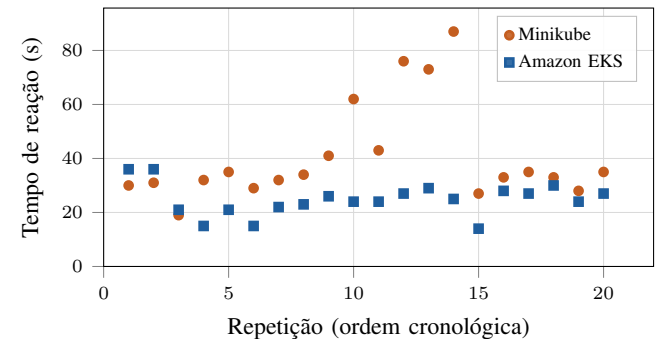


Figura 3. Tempo de reação de cada repetição em ordem cronológica. Uma tendência ao longo da abscissa indicaria que as repetições não são identicamente distribuídas; nenhuma é aparente.

experimento de execução única neste sistema, portanto, erra a direção do efeito em aproximadamente uma vez a cada oito.

Esse número já explica a irreprodutibilidade do estudo anterior, mas as observações individuais podem ser localizadas com precisão ainda maior. A Tabela VI situa cada valor previamente relatado em seu percentil empírico dentro da distribuição aqui medida.

O padrão está completo. A execução E1 por acaso sorteou

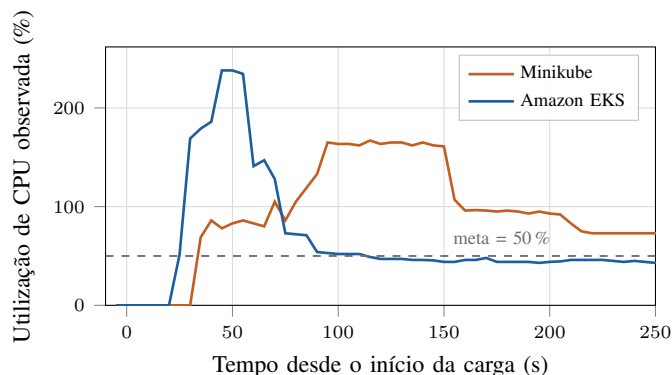


Figura 4. Utilização mediana de CPU observada pelo HPA ao longo das repetições. A variável controlada converge para a meta configurada nos dois ambientes.

Tabela VI

ONDE AS OBSERVAÇÕES ÚNICAS DO ESTUDO ANTERIOR CAEM DENTRO DAS DISTRIBUIÇÕES MEDIDAS NESTA CAMPANHA.

Execução anterior	Ambiente	Relatado	Percentil
E1 (Andrade)	Minikube	62 s	82 %
E1 (Andrade)	EKS	15 s	10 %
E2 (Miqueletti)	Minikube	33 s	45 %
E2 (Miqueletti)	EKS	60 s	100 %

uma observação local lenta (percentil 82) junto de uma observação de nuvem rápida (percentil 10), e assim relatou uma vantagem de quatro vezes para a nuvem — exagerando um efeito real, porém muito menor. A execução E2 sorteou uma observação local absolutamente ordinária (percentil 45) junto de uma observação de nuvem acima de qualquer coisa que registramos em vinte repetições, e assim inverteu a conclusão. Nenhuma das duas execuções foi defeituosa; ambas foram sorteios únicos de distribuições que se sobrepõem.

Uma ressalva se aplica à última linha. O estudo anterior media T_0 na submissão do gerador de carga pelo operador, embutindo o escalonamento e a partida do Pod, que aqui medimos em 1,8 s na nuvem. Isso infla ligeiramente os valores antigos e não explica, por si só, uma observação de 60 s exceder nosso máximo de 36 s; a explicação mais provável é que a instrumentação mais grosseira e manual de E2 tenha superestimado o valor. Trata-se da mesma ameaça de instrumentação heterogênea que motivou o desenho atual.

VII. DISCUSSÃO

A. O que replica e o que não replica

Os resultados se separam nitidamente em dois grupos, e a separação não é a que o estudo anterior propunha.

Replica tudo o que é determinado por declaração. Ambos os ambientes convergiram, ambos estabilizaram a utilização média na meta configurada de 50 %, e ambos descenderam conforme a cláusula `behavior`, com tempos de descida cujos desvios-padrão foram de 6,9 s e 10,7 s sobre médias de 453 s e 389 s — variação de cerca de 2 %. A Equação (1) não contém termo de infraestrutura, e as medições se comportam de acordo. Quem

quer saber se um manifesto *está correto* pode responder isso localmente.

Não replica a temporização, mas a formulação anterior desse fato estava errada. O tempo de reação não é irreprodutível: é uma variável aleatória com distribuição estável, e essas distribuições diferem entre ambientes por uma margem grande e significativa. O que é irreprodutível é um *sorteio único* dela. A distinção importa porque muda o remédio: a resposta não é um experimento único melhor controlado, é repetição.

B. Por que o ambiente gerenciado reage mais rápido

Nossos dados sustentam o efeito, mas não isolam seu mecanismo, e tomamos cuidado para não afirmar mais do que se pode. O candidato óbvio — que o caminho de armazenamento do *control plane* local é duas ordens de magnitude mais lento — é contrariado pela análise de correlação: dentro do ambiente local, as repetições com pior latência de escrita não foram as mais lentas. O que produz o deslocamento parece agir de forma uniforme, e não através dos episódios de degradação de armazenamento que capturamos.

Uma explicação plausível, consistente com tudo o que foi medido, é que o *control plane* gerenciado executa sua cadeia de métricas e sua malha de controle sobre infraestrutura dedicada e consistentemente provisionada, de modo que os *períodos* dessas malhas são honrados com precisão. O *control plane* local compartilha CPU e um caminho de E/S saturado com o hospedeiro, de modo que uma coleta ou uma sincronização pode escapar de seu período nominal. Um escorregamento desse tipo deslocaria a distribuição inteira e alongaria sua cauda, que é o que observamos: quatro repetições locais além de 60 s contra um máximo de 36 s na nuvem. Testar isso adequadamente exigiria instrumentar os tempos internos da malha do controlador, que o serviço gerenciado não expõe.

C. Implicações práticas

Três recomendações decorrem diretamente das medições.

Primeira: **valide a configuração de autoescalamento localmente e meça sua temporização no ambiente de destino.** A documentação da própria AWS para o EKS [5] descreve a configuração, mas nada diz sobre a latência esperada, que, a julgar por nossos resultados, não poderia de todo modo ser declarada como um número único. O *cluster* local reproduziu o resultado lógico com fidelidade e baixo custo. Não reproduziu a latência, e nenhum cuidado adicional numa execução local única teria revelado isso.

Segunda: **não calibre parâmetros de temporização a partir de uma observação única.** Um operador que medisse 62 s no local e 15 s na nuvem — ambos valores que observamos — concluiria que a plataforma responde por uma diferença de quatro vezes. A diferença real de medianas é de 9 s. Dimensionar uma janela de estabilização ou um limiar de alerta a partir de uma execução única corre o risco de codificar um artefato de fase.

Terceira: **relate distribuições, não valores pontuais.** O custo da campanha de vinte repetições aqui relatada foi de cerca de três horas de execução não supervisionada por

ambiente e, a preços de tabela, menos de um dólar americano de consumo de nuvem. Contra isso, o custo de publicar uma conclusão invertida é considerável, e nosso estudo anterior chegou a uma execução de distância de fazer exatamente isso.

D. Custo e assimetria operacional

A campanha de nuvem manteve um *cluster* vivo por cerca de quatro horas e meia, incluindo um reinício. Reportamos o custo com uma distinção que costuma ser omitida: o custo *faturado*, consultado após o fato na API de gestão de custos da conta, foi zero — a conta usada neste trabalho não é cobrada por uso de EKS, EC2 ou VPC — enquanto o custo *a preço de tabela* do mesmo consumo, às tarifas públicas sob demanda de um *control plane*, duas instâncias `t3.small` e um *NAT gateway*, é de aproximadamente US\$ 0,90. Apenas o segundo é informativo para quem planeja uma campanha semelhante, e é uma estimativa, não uma medição. A campanha local não custou nada sob nenhuma das duas contabilidades e levou aproximadamente o mesmo tempo de parede.

A assimetria que importa não é o preço, mas a forma do modo de falha: o ambiente local falhou por degradação — durante a preparação seu *control plane* tornou-se inacessível, com o `etcd` reportando operações de `fdatasync` de 1,9s — e exigiu um guarda de saúde entre repetições para sobreviver a uma campanha não supervisionada. O ambiente gerenciado nunca precisou disso. Nenhuma das campanhas perdeu repetições, mas apenas uma delas precisou da rede de segurança.

E. Generalidade do resultado metodológico

Nada no argumento da Seção VI-F é específico do HPA. Ele exige apenas que a grandeza de interesse seja uma variável aleatória com distribuições sobrepostas entre condições, e que o experimento a amostrasse uma vez por condição. Isso descreve uma fração grande das comparações de infraestrutura de nuvem feitas por profissionais, onde replicar é barato, a variabilidade é alta e comparações de execução única são a norma. A lição transferível é a aritmética: quando $A_{12} = 0,87$, uma execução por condição inverte a ordenação uma vez a cada oito, e o experimento não oferece nenhum sinal de que isso aconteceu.

VIII. AMEAÇAS À VALIDADE

A. Validade de construção

a) *O tempo de reação mede a decisão, não o efeito:* Nossa métrica primária termina na decisão de escala do controlador. O usuário só sente alívio quando novos Pods estão servindo tráfego, o que acrescenta atrasos de escalonamento, de imagem e de prontidão. Reportamos a latência de partida do gerador e o tempo até o regime permanente separadamente precisamente porque esses custos de atuação são onde os ambientes podem legitimamente diferir, mas um leitor interessado em elasticidade de ponta a ponta deve combinar as duas coisas em vez de ler apenas o tempo de reação.

b) *Resolução de um segundo nos carimbos do cluster:* Tanto `startedAt` quanto `lastScaleTime` são carimbos RFC 3339 com granularidade de segundo, de modo que todo tempo de reação carrega até ± 1 s de quantização, além da quantização muito mais grosseira imposta pelo período de 15 s do controlador. Isso afeta de forma desprezível as estimativas de posição, mas infla o número de empates exatos, razão pela qual a correção de empates no teste de Mann–Whitney não é opcional aqui.

B. Validade interna

a) *Repetições dentro de uma única sessão:* Cada campanha roda como uma única sessão ininterrupta em um único *cluster*. Efeitos que derivam ao longo de horas — aquecimento do cache de imagens, comportamento térmico do hospedeiro, atividade de vizinhos do lado da AWS — ficam, portanto, confundidos com o índice da repetição em vez de serem promediados. Mitigamos isso verificando um estado de base explícito antes de cada repetição e publicando o tempo de reação contra a ordem cronológica (Fig. 3), de modo que uma tendência ficaria visível em vez de escondida dentro de um agregado.

b) *Descida forçada na maioria das repetições:* Apenas as três primeiras repetições de cada campanha observam a descida natural; as demais forçam o retorno a uma réplica. As estatísticas de descida, portanto, repousam sobre $n = 3$ por ambiente e são reportadas como secundárias e indicativas, não como distribuição caracterizada. O próprio ato de forçar não pode contaminar o tempo de reação da repetição seguinte, porque a verificação do estado de base exige que a CPU medida tenha decaído antes de a próxima repetição começar.

c) *O ambiente local compartilha seu hospedeiro:* O *cluster* Minikube disputa CPU com o sistema operacional do hospedeiro e com o próprio cliente de medição. Isso não é um defeito a ser eliminado por engenharia: é a condição real de um *cluster* local de desenvolvimento, e faz parte daquilo de que a comparação trata. Significa, porém, que os resultados locais caracterizam *este* hospedeiro, e não Kubernetes local em geral.

C. Validade externa

a) *Uma carga, um gerador fraco:* Um único laço sequencial de `busybox` nunca levou nenhum dos *clusters* ao teto de dez réplicas. A campanha caracteriza o comportamento do controlador em regime de convergência, não os limites de capacidade de nenhum dos ambientes, e nada diz sobre como o tempo de reação se comporta quando o *cluster* está próximo da saturação ou quando Pods ficam `Pending` por falta de capacidade de nó.

b) *Uma topologia, uma região, um tipo de instância:* O braço de nuvem são duas instâncias `t3.small` em `us-east-1`. Instâncias burstáveis têm dinâmica de créditos de CPU que famílias maiores não têm, e uma única região, em uma única conta, em um único dia, é uma fatia estreita de um serviço gerenciado. Escolhemos essa topologia porque é a que o estudo anterior usou, e a comparabilidade com E1 e E2 era o ponto.

c) *Ajuste padrão do controlador*: O período de sincronização do HPA e a resolução do *Metrics Server* foram mantidos em seus valores padrão, e são os termos dominantes no orçamento de latência da Seção II-B. Conclusões sobre tempo de reação são conclusões sobre um HPA *com ajuste padrão*; um operador que reduza o período de sincronização estaria medindo um sistema diferente.

D. Validade de conclusão

a) *Tamanho da amostra*: Vinte repetições por ambiente sustentam uma descrição distribucional e um teste não paramétrico, mas estimam mal as caudas. As afirmações deste artigo são sobre tendência central, dispersão e ordenação estocástica; não fazemos nenhuma afirmação sobre percentis extremos.

b) *Comparações múltiplas*: Várias métricas são comparadas entre ambientes. A análise confirmatória é a declarada em QP1 e QP2 — posição e dispersão do tempo de reação; toda outra comparação é exploratória e deve ser lida como tal, e não como achado significativo independente.

IX. CONCLUSÃO

Este artigo reexaminou uma questão que nosso próprio trabalho anterior havia respondido incorretamente: se o tempo de reação do *Horizontal Pod Autoscaler* do Kubernetes é uma propriedade da plataforma. Aquele trabalho implantou manifestos idênticos em Minikube e Amazon EKS, executou cada implantação uma vez por autor, e constatou que a ordenação dos dois ambientes se invertia entre execuções — de onde concluiu que a diferença não era atribuível à plataforma. Aqui mantivemos a implantação, a carga e o protocolo inalterados, mudamos o desenho de uma para vinte execuções por ambiente, e derivamos a métrica primária de carimbos de tempo atribuídos pelo *cluster* em vez de pelo observador.

Respondendo às questões de pesquisa: (QP1) o tempo de reação é uma variável aleatória cujas distribuições diferem significativamente entre ambientes — média de 40,8 s no local contra 24,7 s na nuvem, $U = 52,0$, $p = 1 \times 10^{-4}$, δ de Cliff = 0,74 — de modo que existe um efeito de plataforma, e ele favorece o serviço gerenciado; (QP2) o ambiente local produziu uma longa cauda à direita que o gerenciado nunca produziu, mas um teste robusto de dispersão não estabelece diferença no espalhamento típico; (QP3) uma execução única por ambiente reproduz a ordenação correta em apenas 87 % das vezes, e as duas observações anteriores situam-se nos percentis 82 e 10 em uma execução e 45 e 100 na outra, o que explica integralmente a inversão; (QP4) todo resultado fixado por declaração — convergência, meta de utilização, descida assimétrica — replicou entre ambientes com variação de poucos por cento.

A correção à nossa conclusão anterior merece ser enunciada com precisão, porque a formulação anterior não era apenas imprecisa, era enganosa. Não é o caso de a latência de autoescalamamento não se relacionar com a plataforma. É o caso de a diferença ser menor do que um único par de observações sugere, e de o ruído em torno dela ser grande o bastante para

inverter a ordenação com frequência. Um resultado negativo obtido com $n = 1$ por condição carrega essencialmente nenhuma informação, e apresentá-lo como achado — como fizemos — afirma mais do que os dados sustentavam.

Os resultados de instrumentação são transferíveis além deste experimento. Derivar a latência de `lastScaleTime` e do `startedAt` do contêiner remove inteiramente o relógio do observador da medição; medir as latências de leitura e de escrita do *API server* como um par isola o caminho de armazenamento do *control plane* da distância de rede, que a latência de escrita bruta confunde tão gravemente que chega a inverter a ordenação; e substituir o *exec credential plugin* do EKS por um *token* renovado foi necessário para amostrar os dois ambientes na mesma taxa.

Trabalhos futuros seguem as limitações, e não os resultados. A extensão mais valiosa instrumentaria os tempos internos da malha do controlador para testar a hipótese de fase diretamente, em vez de inferi-la da ausência de correlação com todas as covariáveis que conseguimos observar de fora. Um gerador de carga paralelo caracterizaria o comportamento próximo ao teto de réplicas, onde o *Cluster Autoscaler* se torna necessário e o tempo de reação pode se comportar de outra forma. Uma campanha que se estendesse por vários dias e famílias de instância separaria efeitos da plataforma gerenciada de efeitos de uma conta em uma tarde. Por fim, o mesmo desenho de medidas repetidas aplicado ao KEDA e ao autoescalamamento por métricas customizadas estabeleceria se a razão entre reprodutibilidade declarativa e variabilidade temporal aqui encontrada é propriedade geral das malhas de controle do Kubernetes.

REPRODUTIBILIDADE

Os manifestos, os *scripts* de campanha, as séries temporais brutas de todas as quarenta repetições, os registros de eventos do controlador, a análise estatística e o código-fonte deste artigo estão disponíveis no repositório do projeto. A análise usa apenas a biblioteca padrão do Python e uma semente aleatória fixa, de modo que cada figura e cada número das tabelas se regeneram a partir dos dados brutos com um único comando. As figuras deste artigo são produzidas pelo PGFPLOTS lendo diretamente a saída da análise, sem arquivos de imagem intermediários.

REFERÊNCIAS

- [1] P. Mell and T. Grance, *The NIST Definition of Cloud Computing*, NIST Special Publication 800-145. Gaithersburg, MD: National Institute of Standards and Technology, 2011.
- [2] Kubernetes, *HorizontalPodAutoscaler Walkthrough*. [Online]. Available: <https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale-walkthrough/>
- [3] Kubernetes, *Horizontal Pod Autoscaling — Algorithm Details*. [Online]. Available: <https://kubernetes.io/docs/tasks/run-application/horizontal-pod-autoscale/>
- [4] Kubernetes SIG, *Metrics Server*. [Online]. Available: <https://github.com/kubernetes-sigs/metrics-server>
- [5] Amazon Web Services, *Scale Pod Deployments with Horizontal Pod Autoscaler*, Amazon EKS User Guide. [Online]. Available: <https://docs.aws.amazon.com/eks/latest/userguide/horizontal-pod-autoscaler.html>
- [6] *eksctl — The Official CLI for Amazon EKS*. [Online]. Available: <https://eksctl.io/>

- [7] T. Mytkowicz, A. Diwan, M. Hauswirth, and P. F. Sweeney, "Producing wrong data without doing anything obviously wrong!" in *Proc. ASPLOS*, 2009, pp. 265–276.
- [8] T. Hoefler and R. Belli, "Scientific benchmarking of parallel computing systems," in *Proc. SC*, 2015, pp. 1–12.
- [9] A. Arcuri and L. Briand, "A Hitchhiker's guide to statistical tests for assessing randomized algorithms in software engineering," *Softw. Test. Verif. Reliab.*, vol. 24, no. 3, pp. 219–250, 2014.